Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Factory Method kalıbı (ekrandan, 00:20:00 ve 00:23:05)
  - Amaç: tek bir nesnenin oluşturulması için bir arayüz tanımlamak, hangi sınıftan nesne oluşturulacağına alt sınıfların karar vermesini sağlamak.
  - Örnek sorun: farklı belge türleriyle çalışan uygulamalar geliştirmeye yarayan bir çerçeve program (framework). Çerçevenin `Application` sınıfı, kullanıcı programcının türeteceği belge sınıflarını önceden bilemez.
  - Örnek çözüm: soyut `Document` (soyut `Open()`, `Close()`; somut `Save()`, `Revert()`) ve soyut `Application` (soyut `CreateDocument()`; somut `NewDocument()`, `OpenDocument()`). `NewDocument()` içinde `Document* doc = CreateDocument(); docs.Add(doc); doc->Open();` çalışır; `MyApplication.CreateDocument()` ise `return new MyDocument` yapar.
  - Kalıp yapısı: soyut `Product`, `ConcreteProduct`; soyut `Creator` (soyut `FactoryMethod()`, somut `AnOperation()` içinde `product = FactoryMethod()`), `ConcreteCreator` (`FactoryMethod()` içinde `return new ConcreteProduct`).
  - Creator'dan Product'a olan çoklu sahiplik ilişkisi kalıbın gereği değildir, örnek problemin gereğidir. Kalıbın özü: soyut bir oluşturucu sınıf, onun soyut fabrika metodu ve bu metodu çağıran en az bir somut metot.
  - Uygulanabileceği anlar: bir sınıf nesne oluşturmalı ama nesnenin türü kodlama aşamasında belli değilse; bir sınıf sorumluluğunu yardımcı alt sınıflara iletecekse ve hangisinin oluşturulacağı o anki duruma bağlıysa.
  - İkinci örnek (ekrandan, 00:27:20): çizim uygulamasında fareyle değiştirilen şekiller. `Figure.CreateManipulator()` bir fabrika metodudur; `LineFigure` bir `LineManipulator`, `TextFigure` bir `TextManipulator` oluşturur. Paralel iki sınıf hiyerarşisi ortaya çıkar.
- Factory Method örneği: e-ticaret sitesi (ekrandan, 00:40:00 ve 00:41:15)
  - Her dükkân tek tür ürünle çalışır; sipariş işleme adımlarının sırası aynı, iç mantığı ürün türüne göre çok farklıdır.
  - Soyut `Store` sınıfı (Creator): `protected abstract Order createOrder(String uniqueID)` ve `public void makeOrder(String uniqueID)`. `makeOrder` siparişi oluşturur, sonra `prepare()`, `orderFromSupplier()`, `pack()`, `sendToCustomer()` adımlarını sırayla çağırır.
  - Soyut `Order` (Product, bütün metotları soyut); somut ürünler `Electronics`, `Books`; somut oluşturucular `ElectronicsStore`, `BookStore`.
  - Yeni bir ürün türü (örneğin tekstil) için yalnız iki sınıf eklenir: `Store`'dan türeyen bir dükkân ve `Order`'dan türeyen bir ürün.
- Builder kalıbı (ekrandan, 01:00:00, 01:01:00, 01:04:30)
  - Amaç: karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmak; böylece aynı oluşturma süreci farklı gösterimler üretebilir.
  - Örnek: RTF okuyan ve metni farklı biçimlere (ASCII, TeX, düzenlenebilir metin bileşeni) çeviren program. `RTFReader.ParseRTF()` her token için türüne göre `ConvertCharacter`, `ConvertFontChange`, `ConvertParagraph` çağırır; soyut `TextConverter` alt sınıfları `ASCIIConverter`, `TeXConverter`, `TextWidgetConverter`dır ve ürünleri `ASCIIText`, `TeXText`, `TextWidget`tir.
  - Kalıp yapısı: `Director` (`Construct()`), soyut `Builder` (`BuildPart()`), `ConcreteBuilder` (`BuildPart()`, `GetResult()`), `Product`.
  - Etkileşim (sıra şeması): istemci somut builder'ı oluşturur, onu `Director`'a verir,

`Construct()` der; Director `BuildPartA/B/C()` çağrılarını yönetir; istemci sonucu `GetResult()` ile builder'dan alır.
  - Uygulanabileceği anlar (ekrandan, 01:10:00): karmaşık nesneyi oluşturma algoritmasının parçalardan ve parçaların birleştirilme biçiminden bağımsız olması gerektiğinde; oluşturma sürecinin nesnenin farklı gösterimlerine izin vermesi gerektiğinde.
  - Sonuçlar: ürünün iç yapısı en az yan etkiyle değiştirilebilir (yeni bir ConcreteBuilder yazarak); fabrika kalıplarına göre ürün oluşturma süreci üzerinde daha fazla denetim vardır (Director, Builder'a hangi işlemi hangi sırada yapacağını söyler).
- Builder örneği: otomobil adlarını çözümleyen modül (ekrandan, 01:20:00 ve 01:28:10)
  - Farklı markaların model adları (kasa tipi, motor, vites bilgisi) farklı biçimde yazılır. Ad, anlamı olan parçalardan oluşan karmaşık bir üründür.
  - `CarDirector` (`builder: CarBuilder`, `construct(elements[]: String)`, `report()`); soyut `CarBuilder` (`product: Car`, `getProduct()` protected, `getResult()`, soyut `buildChassis()`, `buildEngine(token)`, `buildTransmission(token)`); somut `RenaultBuilder` ve `HondaBuilder`; ürün `Car`.
  - Kasa türleri sınıf olan (primitif olmayan) bir Java enum'u ile tutulur; parantez içindeki değerler enum kurucusuna verilen parametrelerdir.
  - Her marka kendi kurallarıyla adı çözümler: örneğin Honda'da motor bilgisinde "ECO" geçiyorsa yakıt LPG'dir, "PS" gücü verir, vites bilgisinde "AT" otomatik demektir; Renault'da "DCI" yakıt türünü, "BG" gücü, "CVT" ya da "EDC" otomatik vitesi belirtir.
- Prototype kalıbı (ekrandan, 01:50:00, 02:00:00, 02:10:00)
  - Amaç: oluşturulacak nesnelerin türlerini bir prototip nesneyle belirlemek ve yeni nesneleri bu prototipi kopyalayarak oluşturmak.
  - Örnek: hazır bir grafik çerçeve programıyla müzik besteleme programı yazmak. Çerçevede soyut `Tool` (`Manipulate()`), `RotateTool`, `GraphicTool` ve soyut `Graphic` (`Draw(Position)`, `Clone()`) vardır. Uygulamada `Staff` (porte), soyut `MusicalNote`, `WholeNote`, `HalfNote` sınıfları `Clone()` metodunda kendi kopyalarını döndürür.
  - `GraphicTool.Manipulate()`: `p = prototype->Clone(); while (user drags mouse) { p->Draw(new position) } insert p into drawing`.
  - Kalıp yapısı: `Client` (`Operation()` içinde `p = prototype->Clone()`), soyut `Prototype` (`Clone()`), `ConcretePrototype1/2` (`return copy of self`).
  - Uygulanabileceği anlar (ekrandan, 02:20:00): sistem ürünlerin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olmalıysa, ve ek olarak oluşturulacak sınıflar çalışma anında belirlenecekse, ürün hiyerarşisine paralel bir factory hiyerarşisi kurulmak istenmiyorsa ya da ürün sınıfının örneklerinin durum uzayı sınırlıysa. Son üç koşul kalıbı Factory Method'dan ayırır.
  - Kalıp C++ gibi sınıfların doğrudan işlenemediği dillerde daha yararlıdır; SmallTalk gibi geç ilişkilendirme (late binding) ya da Java gibi yansıtma (reflection) yetenekleri olan dillerde kalıp yerine dilin yetenekleri kullanılabilir.
- Prototype örneği: oyun için bölüm (level) düzenleyicisi (ekrandan, 02:36:10 ve 02:40:00)
  - Soyut `Monster` sınıfı `HP`, `DEX`, `AGL`, `LUCK`, `CHR` ve `type` alanlarını, soyut `clone()` metodunu, public getter'ları ve protected setter'ları içerir. `GenericMonster` somut prototiptir; `MonsterRegistry` bir `HashMap` içinde canavarları türlerine göre saklar (`findMonster`, `registerMonster`).
  - Yeni bir canavar türü, kayıtlı bir canavarı klonlayıp birkaç alanını değiştirerek ve yeni adla kaydederek oluşturulur.
- Singleton kalıbı (ekrandan, 02:45:30 ve 02:50:00)
  - Amaç: bir sınıfın yalnız tek bir örneğinin bulunmasını sağlamak ve bu örneğe bütün istemcilerin tek bir noktadan erişmesini sağlamak.
  - Örnekler: bir işletim sisteminde tek yazdırma kuyruğu (print spooler) süreci, tek pencere yöneticisi, tek çekirdek süreci.
  - Çözüm: tek olma sorumluluğu sınıfın kendisine verilir. Sınıfın kendi türünden static bir üyesi (`uniqueInstance`) olur, kurucusu private olur, bu üye için static bir erişim metodu (`getInstance()`) bulunur: `if (uniqueInstance == null) { uniqueInstance = new Singleton(...) } return uniqueInstance;`.
  - Ne zaman: bazı nesnelerin türünün tek örneği olması gerektiğinde ve bu örneğin birçok farklı sınıfın nesneleri tarafından kullanılması istendiğinde.

Hocanın Özellikle Vurguladığı Kısımlar

- Abstract Factory ile Factory Method'u karıştırmayın
  - İkisinin adında da "Factory" geçer ama çözdükleri sorunlar farklıdır. Abstract Factory bir ürün ailesinden nesne üretir; Factory Method tek bir nesneyi oluşturur ve sınıfına alt sınıflar karar verir.
  - E-ticaret örneğinde ürün ailesi yoktur; her dükkân tek tür ürünle çalışır. Bu yüzden Factory Method uygundur; hocanın deyişiyle iki kalıp arasında "dağlar kadar fark" vardır.
- Çizimler güncel UML standardına uymalı
  - Soyut sınıflar yalnız italik yazılmakla kalmaz, sınıf adının üstünde «abstract» stereotipi de yazılır. Dörtlü Çete'nin eski gösteriminde kalıtım okunun ucu çizginin ortasına konmuştur; güncel standartta ok ucu üst sınıfa değecek biçimde çizilir.
  - `clone()` gibi kendi türünden nesne döndüren metotlarda dönüş türü de gösterilmelidir (örneğin `Clone() : Graphic`).
  - Bağımlılık okları (kesik çizgili oklar) açıkça çizilmelidir: bir sınıf başka bir türü parametre alıyor, yerel olarak oluşturuyor ya da döndürüyorsa bu ok gösterilir. Az ok görüp bağlaşımı olduğundan düşük değerlendirmemek gerekir.
  - Sınavlardaki UML şemalarının kurallara ve güncel standarda uyması değerlendirmenin bir bileşenidir.
- Builder'ın zayıf yönleri (ekrandan, 01:00:00)
  - Çok biçimlilik tam kullanılamaz; ayrıştırıcıda (Director) switch-case hâlâ durur. Yeni bir bileşen türü için hem bir case hem üst sınıfa soyut metot hem de alt sınıflarda gövde eklemek gerekir.
  - Alt sınıflar üst sınıfın bütün metotlarını gerçeklemek zorundadır; anlamı olmayan metotlar boş gövdeli kalır (örneğin düz metinde yazı tipi değişikliği).
  - Sınıf şemasında switch-case görünmez ama `ConcreteBuilder.buildPart` içinde kendini gösterir (ekrandan, 01:01:00).
- Prototype'ta sığ ve derin kopyalama
  - Sığ kopyalama (shallow copy) yalnız nesnenin kendisini kopyalar, primitif olmayan üye alanlarının gösterdiği nesneleri kopyalamaz; hızlıdır ama ilişkiler kopyaya taşınmaz.
  - Derin kopyalama (deep copy) ilişkili nesneleri de zincirleme kopyalar. İlişkilerde çevrim varsa sonsuz döngü gibi sorunlara dikkat etmek gerekir.
- Kalıbı ihtiyaca göre uyarlayın
  - Somut alt sınıflara ihtiyaç yoksa Prototype sınıfı tek bir somut sınıf da olabilir; `GenericMonster` alt sınıfının gerekliliği tartışılabilir. Bir kalıbın bazı kısımları gereksinime göre silinebilir ya da değiştirilebilir.
- Singleton'ın erişim metodu Factory Method kalıbı değildir
  - Slaytta "factory metodu" denen erişim metodu, paylaşılan örneği döndüren bir getter gibi düşünülmelidir. Adının `getInstance()` konması, fabrika kalıplarıyla karışmasını önler.
  - Tek örnek yalnız bir sınıf tarafından kullanılacaksa kalıba gerek yoktur; o sınıfta static bir üye alan yeterlidir.

Kısa Tekrar Notları

- Factory Method: soyut Creator'da soyut fabrika metodu; somut Creator hangi ConcreteProduct'ın oluşturulacağına karar verir. Tek bir ürün, aile yok.
- Abstract Factory: ilişkili ürün ailesinden nesneler. Builder: parçalardan oluşan karmaşık ürün, oluşturma süreci üzerinde adım adım denetim.
- Factory Method'da soyut ve somut metotlar birlikte bulunur: çerçevenin yapabildiği işler somut, uygulamaya bağlı işler soyut metottur.
- Java'daki generic sınıflar ya da C++ template'leri benzer sorunlara bir çözüm olabilir, ama dile özeldir; taşınabilirlik açısından zayıftır.
- Ortak adımlar aynı, yalnız birkaç adım farklıysa ileride görülecek Template Method daha uygun olabilir; adımların iç mantığı tamamen farklıysa Factory Method.
- Builder bileşenleri: Director (yönlendirir, kendisi yapmaz), Builder (soyut parça oluşturma arayüzü), ConcreteBuilder (parçaları oluşturur, birleştirir, sonucu verir), Product.
- Director ile Builder arasındaki ilişkiyi istemci kurar: uygun ConcreteBuilder'ı seçip Di-

rector'a verir.
- Builder'da yeni hedef biçim (örneğin PDF) için yalnız yeni bir `TextConverter` alt sınıfı yazılır.
- Prototype: yeni nesne sıfırdan değil, bir prototipin `clone()` ile kopyalanmasıyla oluşturulur.
- Prototype'ın Factory Method'a göre artısı: çalışma anında belirlenen sınıflar, paralel factory hiyerarşisine gerek olmaması, daha az sınıf.
- Kardeş alt sınıfların birbirinden haberdar olması normalde istenmez; Prototype'ta bir `clone()` gerekiyorsa kardeş türden nesne de döndürebilir.
- Bir alt sınıfa özgü metoda üst tür üzerinden erişmek tür dönüşümü (type casting) gerektirir; bu iyi bir durum değildir.
- Singleton: kendi türünden static üye, private kurucu, static `getInstance()`. Setter eklenebilir, eklenmeyebilir.
- Strategy ve State gerçeklemeleri iyi birer Singleton adayıdır. Singleton sınıfları kalıtımla özelleştirmek, static üyeyi kullanıcı sınıflardan birine koymaktan daha kolaydır.
- Tek nesneyi kullanıcı sınıflardan birinin static üyesi yapmak Feature Envy kod kusuruna yol açabilir; düzeltmek için Move Field yapılır.

Detaylı Açıklamalar

Factory Method'un çerçeve program mantığı

Factory Method'un örnek sorununda iki temel varlık vardır: üzerinde çalışılacak belgeler ve hazırlanacak uygulama. Çerçeve programın yazarı, bu iki temel sınıftan kalıtımla kullanıcı programcıların kendi alt sınıflarını üretmesini ister. Hoca şemayı yatay bir çizgiyle ikiye böldü: üst kısım çerçeve, alt kısım çerçeveyi kullanan uygulamadır (ekrandan, 00:20:00).

`Document` sınıfında soyut ve somut metotlar birlikte bulunur. Bir dosyayı açıp baytlarını okumak her dosya türünde aynıdır, ama dosyanın mantıksal içeriğine göre ek işlemler gerekebilir; kapatmadan önce tutarlılık denetimi de uygulamaya göre değişir. Kaydetme ise ikilik kipte bilgi aktarımıdır ve çerçeve bunu somut metot olarak kodlayabilir. Geri alma işlemleri ileride Memento kalıbıyla ayrıca modellenecektir. `Application` sınıfında da yeni belge oluşturmanın genel kısmı (kayıt iletişim kutusu, yeni sekme ya da pencere açmak) çerçevenin yapabileceği bir iştir; belgenin gerçekten oluşturulması ise belge türüne bağlıdır ve soyut `CreateDocument()` metoduna bırakılır. Temel fikir: yapılabilecek kısımları somut metotlarda yap, uygulamaya bağlı kısımlar için soyut metotlar tanımla ve kodu bu soyut metotlar üzerinden tamamla.

Çizim uygulaması örneğinde her şekil türü farklı biçimde değiştirilir: bir çizginin uzunluğu, kalınlığı ya da kesikli olması değiştirilebilir; bir metin kutusu en-boy oranı korunarak büyütülebilir ama "kesikli metin" diye bir şey yoktur. Bu yüzden her şekil kendi manipülatörünü oluşturur. Manipülatör sınıfının bütün metotları soyutsa bir Java arayüzü olarak da tanımlanabilir; somut bir metot ya da alt sınıflara aktarılacak bir alan gerekiyorsa soyut sınıf seçilir. Bazı şekiller özel bir manipülatöre ihtiyaç duymuyorsa `Manipulator`'ı somut yapmak kalıbın yapısını bozar; bunun yerine bir `DefaultManipulator` alt sınıfı oluşturmak daha iyidir.

E-ticaret örneği ve `Store` kodu

Hoca örneği Head First Design Patterns kitabından değiştirerek aldığını söyledi. Çalışan, gelen elektronik ürün siparişi için elektronik dükkânına, kitap siparişi için kitap dükkânına tekil tanımlayıcıyla sipariş verir. Kod (ekrandan, 00:41:15):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
```

```
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
public class BookStore extends Store {
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

Hoca soyut metodu protected yapmanın şart olmadığını, kaynak kitaptaki tercihe uyduğunu söyledi. Elektronik üründe garanti belgesinin işlenmesi gibi adımlar kitapta yoktur; iade koşulları da ürün türüne göre değişebilir.

Builder: araç adı örneği

`CarDirector.main` içinde Honda için şu kod çalıştırıldı (ekrandan, 01:28:10):

```
CarDirector director = new CarDirector( new HondaBuilder("Civic Sedan") );
String elements[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
director.construct(elements);
director.report();
```

Her öğe bir sekmeyle ayrılmış iki parçadan oluşur: parça türü ve değeri. Director parça türüne göre ilgili `build...` metodunu çağırır; somut builder da değeri kendi markasının kurallarıyla çözümler. Hoca, string'lerde switch-case yerine if-else zinciri kullandığını, bu zincirin kalıbın standart örneğindeki switch-case'e denk geldiğini belirtti. Program çıktısına göre araç sedan kasalı, benzinli, 1600 cc, 125 beygir gücünde, 6 ileri otomatik vitestir (ekrandan ve derste çalıştırıldı, 01:32:43). Hoca bu kuralların bir kısmının örnek için uydurulduğunu söyledi. Renault'nun adlandırması değişirse yalnız `RenaultBuilder`'ın ilgili kısmı değişir; diğer markaların koduna dokunulmaz.

Prototype: müzik besteleme ve oyun örnekleri

Müzik örneğinde porte beş çizgiden, notalar içi dolu ya da boş elipslerden ve kuyruklardan oluşur; az sayıda temel şekil vardır. Not sürüklenirken her küçük adımda eski kopya silinip yeni konumda yeni bir kopya çizilir; yeni bir satıra porte çizerken de üstteki porte klonlanır. Sıfırdan nesne oluşturmaktansa önceki nesneyi klonlamak daha hızlı ve pratik bir çözümdür.

Hoca problemin Abstract Factory ya da Factory Method ile de çözülebileceğini söyledi. Abstract Factory'de nota türleri (tam, yarım, ters) aile, do'dan si'ye notalar ailenin üyeleri olurdu ve daha çok sınıf gerekirdi. Factory Method'da her nota türü için ayrı bir fabrika sınıfı gerekir. Prototype ise nota türü kadar sınıfla ihtiyaçları daha basit karşılar; Dörtlü Çete de bu örnekte Prototype'ı seçmiştir. Bellek optimizasyonu ise Flyweight kalıbının konusudur.

Oyun örneğinde (Head First Design Patterns'tan) kod şöyledir (ekrandan, 02:36:10 ve 02:40:00):

```
package dp.prototype.example;
public abstract class Monster {
    protected int HP, DEX, AGL, LUCK, CHR;
    protected String type;

    public abstract Monster clone( );

    protected Monster(int hp, int dex, int agl, int luck, int chr) {
        HP = hp; DEX = dex; AGL = agl; LUCK = luck; CHR = chr;
    }
```

```
    public String getType() { return type; }
    public int getHP() { return HP; }
    protected void setHP(int hP) { HP = hP; }
    public int getDEX() { return DEX; }
    protected void setDEX(int dEX) { DEX = dEX; }
    public int getAGL() { return AGL; }
    protected void setAGL(int aGL) { AGL = aGL; }
    public int getLUCK() { return LUCK; }
    protected void setLUCK(int lUCK) { LUCK = lUCK; }
    public int getCHR() { return CHR; }
    protected void setCHR(int cHR) { CHR = cHR; }
    protected void setType(String type) { this.type = type; }
}

public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
            reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bolum sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

Slayttaki notta setter'ların protected olmasıyla erişimin tek yolunun Registry olduğu belirtilir. Hoca bunun alt sınıfların alanları rahatça değiştirebilmesi, başka istemcilerin değiştirememesi için yapıldığını söyledi; Java'da `Monster` ile `MonsterRegistry` aynı pakette olduğu için paket dışından kimse oyunu kendi amacına göre değiştiremez.

Yansıtma ve geç ilişkilendirme

C++ gibi kuvvetli tip denetimli dillerde bir sınıfın kodu yazıldıktan sonra üye alanları ve metotları çalışma anında değişmez. SmallTalk'taki geç ilişkilendirmede ise nesnenin türü ilk atamada belli olur; böyle bir dilde Prototype'a gerek kalmaz. Java'daki Reflection API ile herhangi bir nesnenin sınıfı, üye alanları ve metotları programla sorgulanabilir; Apache'nin BCEL (Bytecode Engineering Library) kütüphanesi bir adım öteye giderek sınıfın bytecode'unu çalışma anında değiştirebilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.